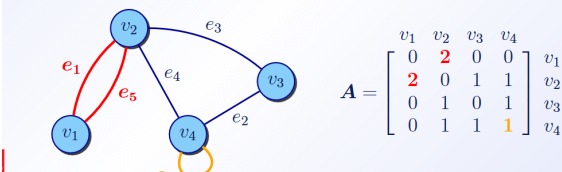


Chap 2 (Représentation des graphe)

Matrice d'adjacence

Matrice qui donne si a et b on une arrette

EXEMPLE.



Matrice d'incidence

c'est matrice qui donne les sommets a qui l'arrette est reliée (col = arret et row = sommet)
(peut pas représenter des boucles lmao)

EXEMPLE.

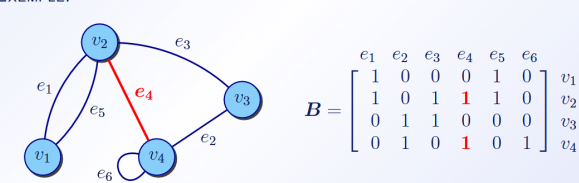


Tableau compact de successeurs (forward star representation)

En gros, c'est 2 tableau :

PS : dans le cas d'un graph non orientier, stocker qu'un fois les arretes

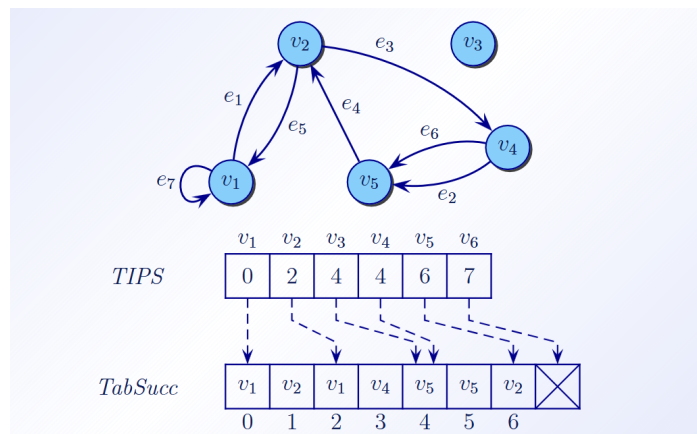
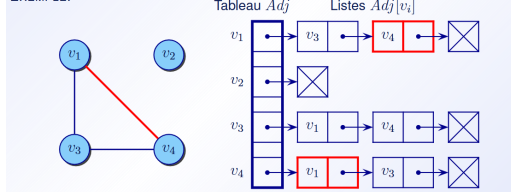


Tableau de lists d'adjacence

c'est un tableau de list chaîner pour montre qu'elle sommet sont accessibles depuis le sommet k

EXEMPLE.



Johnson

En gros, l'idée c'est de battre les circuits négatifs avec une transformation de graph

1. Ajouter un sommet fictif qu'on va appeler O
2. créer des arcs de $O \rightarrow \text{AllVertex}$ de poids 0
3. Bellman ford sur ce nouveau graph avec O comme racine $\rightarrow$ crée le list de distance $d[i]$
4. corriger les arcs du graph en faisant $arc_{ij} + d[i] - d[j]$ (ça va rendre chaque arc non négatif)
5. Dijkstra sur le "nouveau" "nouveau" graph
6. Corriger le résultat de Dijkstra avec le poids réel des arcs

Chap 4 (Arbres et arborescences)

Def

- Arbre = Graph connexe et sans cycles
- Arbre recouvrant = Graph partiel de G qui est un arbre
- coupe = ensemble d'arcs ayant une extrémité dans A et l'autre dans $\bar{A}$

Union-Find

type de structure qui permet de faire les opérations suivantes :

- MakeSet(u) [$O(n)$] : crée un sous-ensemble qui correspond à la condition u
- Find(u) [$O(n)$] : trouve un sous-ensemble contenant u
- Union(u, v) [$O(n)$] : fusionne u et v

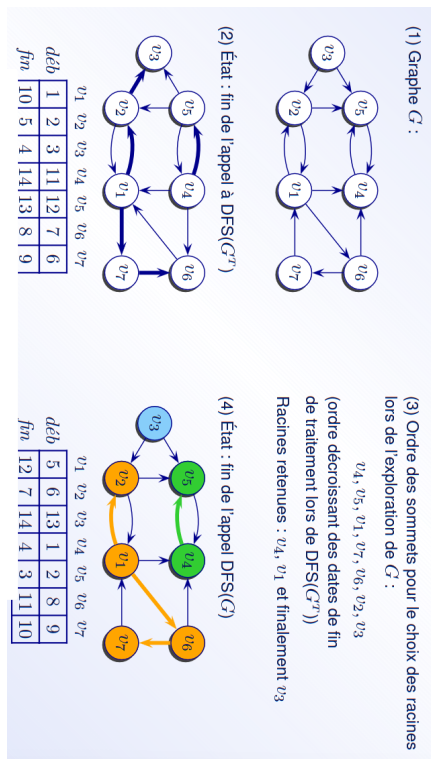
Kruskal découverte d'arbre

Complexité : $O(mn)$ { $O(m \log(n))$: trie, $O(mn)$: parcourt/check}

Algo :

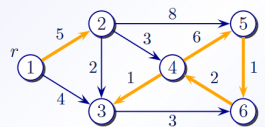
```
// not real c, just c like
int[] kruskal(Graph g){
    int[] fo;
    // trie arrete croissant
    g.arcs.sort(/*By weight*/);

    for(int e : g.arcs){
        // si e ne crée pas de cycle
        // avec fo -> ajouter a la foret
        if(e not cycle in fo){
            fo.push(e);
        }
    }
    return fo;
}
```

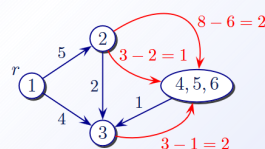


Chu-liu

- (1) Choisir pour chaque sommet (autre que r) l'arc entrant de plus petit poids.



- (2) Contracter chaque circuit créé et mettre à jour le poids des arcs y entrant.



Dijkstra

NE PEUT ÊTRE UTILISER QUE SUR DES CIRCUIT A COUP NON NÉGATIVE

Alog :

```
// g = gaph, s = origine de l'arbre
pair Dijkstra(Graph g, int s){
    // distance du sommet
    int[] dist = int[g.vertices.lenght];

    // plus proche du somette
    int[] proche = new int[g.vertices.lenght];

    // init
    for(uint i = 0; i < dist.lenght; i++){
        dist[i] = INFINIT;
        proche[i] = -1; // NULL
    }

    dist[s] = 0;

    // list des sommettes
    // qui ne sont pas traiter
    int[] buffer = sum;

    while(!buffer.empty()){
        int u = buffer.get(
            sommet avec la plus petit dist[u];
        );

        // cella voudrais dire que
        // que u n'est pas accible
        // et que dont tous les autre
        // sommet restant ne le sont pas
        // non plus
        if(dist[u] == INFINIT) break;

        for (int j : g.get_neighbore(u)){
            if(buffer.include(j) && dist[j] > dist[i] + g.get_arc(i,j).1){
                dist[j] = dist[i] + g.get_arc(i,j).1;
            }
        }
    }
    return {dist,proche};
}
```

Bellman-ford

(k = 0)

(k = 1)

(k = 2)

GAA

GAA

Itér. k	Arc testé	Couples (λ _j , p[j]) pour chaque sommet				Valeur de Continuer
		1	2	3	4	
1	Valeurs de départ	(0,—)	(∞,—)	(∞,—)	(∞,—)	faux
	(1,2)		(1,1)			vrai
	(1,3)			(-3,1)		
	(2,3)					
	(2,4)				(3,2)	
	(3,2)		(0,3)			
	(3,4)				(1,3)	
2	Valeurs de départ	(0,—)	(0,3)	(-3,1)	(1,3)	faux
	(1,2)					
	(1,3)					
	(2,3)					
	(2,4)					
	(3,2)					
	(3,4)					

Prim

(1)

(2)

(3)

(4)

(5)

(6)

Itér.	Sommet retiré de L	Couples (λ[u], p[u]) pour les sommets encore dans L, en fin d'itér.					
		1	2	3	4	5	6
0		(0,—)	(∞,—)	(∞,—)	(∞,—)	(∞,—)	(∞,—)
1	1		(2,1)	(1,1)	(∞,—)	(∞,—)	(∞,—)
2	3		(1,3)		(6,3)	(9,3)	(∞,—)
3	2				(6,3)	(9,3)	(∞,—)
4	4					(5,4)	(3,4)
5	6					(4,6)	
6	5						

Floyd-Warshall

on vas crée une matrice de distance entre chaque sommets ainsi qu'une matrice de prédéceseur pour trouver les chemins le plus court facilement

```
// g = gaph, s = origine de l'arbre
int[][] FloydWarshall(Graph g, int s){
    int[][] dist = int[g.s.lenght][g.s.lenght];
    int[][] proches = int[g.s.lenght][g.s.lenght];

    // init matrice de distance et proches
    for(uint i = 0; i < g.s.lenght; i++){
        for(uint j = 0; j < g.s.lenght; j++){
            dist[i][j] = g.get_arc(i,j).1;
            proches[i][j] = (i == j || mat[i][j] == INFINIT)? NULL : i;
        }
    }

    for(uint k = 0; k < g.s.lenght; k++){
        for(uint i = 0; i < g.s.lenght; i++){
            for(uint j = 0; j < g.s.lenght; j++){
                if(dist[i][j] > dist[i][k] + dist[k][j]){
                    dist[i][j] = dist[i][k] + dist[k][j];
                    proches[i][j] = proches[k][j];
                }
            }
        }
        // graphe continuer cercuit negative
        // passant par i
        if(dist[i][i] < 0) return;
    }

    return {dist,poches};
}
```

$W^{(1)} =$

0	∞	3	0
∞	0	4	8
4	∞	0	11
∞	0	7	0

$P^{(1)} =$

—	—	1	—
—	—	2	—
4	—	1	—
—	—	3	—

Tarjan

dfsnum

1	2	0	4	3	5	6
1	1	—	3	3	1	1
0	0	0	1	1	0	0

low

scc

$N = 6$

$K = 1$

pile P

v7
v6
v2
v1

pile d'appels

SCC(v6)
SCC(v1)

Kurskal

(1)

(2)

(3)

(4)

(5)

(6)

(7)

k	e _k	c(e _k)	∈ T
1	{1,3}	1	✓
2	{2,3}	1	✓
3	{1,2}	2	✓
4	{4,6}	3	✓
5	{5,6}	4	✓
6	{4,5}	5	✓
7	{3,4}	6	✓
8	{2,4}	7	✓
9	{3,5}	9	